

Resource Access Protocols

Raihan Uddin

Electronic Engineering

Hochschule Hamm-Lippstadt

Lippstadt, Germany

raihan.uddin@stud.hshl.de

Abstract—This document is a model and instructions for L^AT_EX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert.

I. INTRODUCTION

According to [1], software synthesis is a process where a customized code is automatically generated from system-level specification. Today, there is always a push for market growth in the segment of Silicon industry. The demand for high performance, low power consumption, and cost-effective solutions is increasing. In quest of embedding many-processor systems on a single chip (MPSoCs), system complexities are growing at an almost exponential rate [2]. The demand for sophisticated high-performance, low-powered processors to meet the system complexities is unprecedented for today's applications. However, the traditional design processes, "in which systems are designed directly at the low hardware or software levels" [2], are fast becoming obsolete and infeasible to bridge the gap "generated by the disparity between the rapid paces at which design complexity has increased in comparison to that of design productivity" [2]. In this context, the software-synthesis approach offers a viable design solution.

Most major semiconductor roadmaps suggest a raise in the level of abstraction in the design process as an acceptable solution for closing the productivity gap [2]. Higher abstraction levels is more aligned with the system requirements and specifications. Software synthesis can bridge the semantic gap between higher abstraction levels and low-level implementation [2]. As stated in [3], software must be developed specifically for the given cores, platform, applications and mapping for Hardware-dependent-Software (HdS). Also described by [3] that the manual HdS development can become time consuming not only due to the code size, but also due to the complex interaction processes in concurrent applications mapped to multi-core platforms. The software synthesis approach enables the automatic generation of code from high-level specifications, thus reducing the complexity and time required for manual coding. More specifically, this is a design methodology that allows designers to apply "design-automation techniques for modeling, simulation, synthesis, and verification to the system-design process" [2].

II. Y-CHART

Y-Chart, developed in 1983, also known as the Gajski-Y-Chart, is a design methodology that provides a framework for the design of complex systems, particularly in the context of hardware and software co-design. According to [2], the Y-Chart approach is characterized by its three main components or axes: behavior (functionality/specification), design structure (netlist/block diagram), and physical design (layout/board design). These components are represented in a Y-shaped diagram, where each arm of the Y represents a different aspect of the design process. The Y-Chart in figure 1 comprises

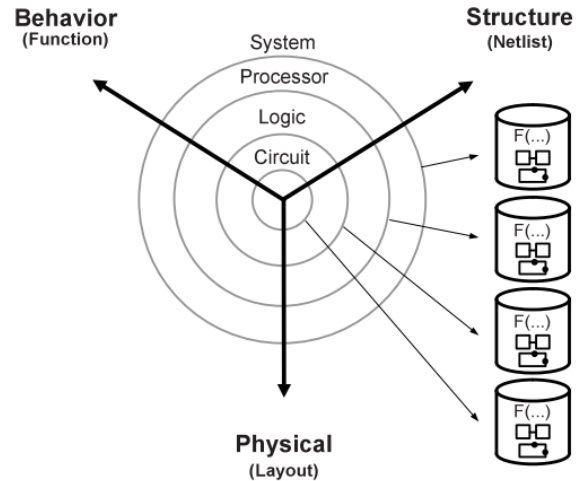


Fig. 1. Y-Chart approach [2].

three axes, each representing a different aspect of every design process [2]. The axes are as follows:

- **Behavior:** "What" the system should perform
- **Structure:** "How" the system is organized
- **Layout:** "Where" the system is implemented

The concentric circles around the origin of the Y-Chart represent design on different abstraction levels and the name of each abstraction level originates from the types of the components generated on that abstraction level [2]. The four levels of abstraction are as following:

- **Circuit level:** The components generated in this level are basic building blocks of the circuit i.e., N-type or P-type transistors.

- **Logic level:** This level utilizes logic gates and flip-flops to generate register-transfer components e.g., registers, register files and also functional units such as ALUs and multipliers [2].
- **Processor level:** At this level, standard and custom processors along with special-hardware components e.g., memory controllers, arbiters, bridges, routers, and various interface components are generated [2].
- **System level:** At this level, embedded systems are designed using processors, memories, buses and other processor components [2], generated on the lower levels.

The process of implementing a given behavior into a virtual structure on each abstraction level is known as synthesis. Each abstraction level should also contain a database of components that are comprising the structure. After synthesizing at one level, one can move to next lower abstraction level [2] and synthesize the structure further. However, if there is predefined library of components available with their structures and physical dimensions, one can proceed with the physical designing that consists of floorplanning, placement and routing on the chip or PC board [2]. Because of the modern day advancement in available software tools, it is possible to start designing or synthesizing directly from the system and processor levels without considering the lower levels. As stated in [2], “every system component except standard processors and memories is synthesized to the logic level, before the physical design is performed on the logic level”. This means that all custom components (e.g., controllers, accelerators etc.) except few are readily available to synthesize to the next lower level of abstraction that is logic level in the next step. These exceptional components- processors and memories are reused directly as IPs. In this regard, it is mentioned in [2] that “some components on the processor-and-system levels may be obtained as IPs and not synthesized. Therefore, their structure and physical design are known, at least partially, on the level higher than logic level. In that case, the physical design then may contain components of different sizes and from different levels of abstraction”.

III. SYNTHESIS AT PROCESSOR LEVELS

IV. SOFTWARE SYNTHESIS PROCESS

V. HARDWARE/SOFTWARE CO-DESIGN

VI. OPTIMIZATION TECHNIQUES

VII. TOOLS AND FRAMEWORKS

VIII. CHALLENGES IN SOFTWARE SYNTHESIS

IX. CASE STUDIES & APPLICATIONS

X. FUTURE TRENDS

REFERENCES

- [1] E. Tavares, B. Silva, P. Maciel, and P. Dallegrave, “Software synthesis for hard real-time embedded systems with energy constraints.” [Online]. Available: <https://ieeexplore.ieee.org/document/4685735>
- [2] D. D. Gajski, S. Abdi, A. Gerstlauer, and G. Schirner, *Embedded System Design: Modeling, Synthesis and Verification*. Springer Science+Business Media, 2009.
- [3] S. Abdi, G. Schirner, H. Cho, Y. Hwang, and L. Yu, “Hardware-dependent software synthesis for many-core embedded systems.” [Online]. Available: <https://ieeexplore.ieee.org/document/4796498>